

SPRING CORE AND MAVEN

Exercise 1: Configuring a Basic Spring Application

Scenario:

Your company is developing a web application for managing a library. You need to use the Spring Framework to handle the backend operations.

Steps:

1. **Set Up a Spring Project:**
 - Create a Maven project named **LibraryManagement**.
 - Add Spring Core dependencies in the **pom.xml** file.
2. **Configure the Application Context:**
 - Create an XML configuration file named **applicationContext.xml** in the **src/main/resources** directory.
 - Define beans for **BookService** and **BookRepository** in the XML file.
3. **Define Service and Repository Classes:**
 - Create a package **com.library.service** and add a class **BookService**.
 - Create a package **com.library.repository** and add a class **BookRepository**.
4. **Run the Application:**
 - Create a main class to load the Spring context and test the configuration.

PROGRAM:

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.library</groupId>
  <artifactId>LibraryManagement</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>5.3.30</version>
    </dependency>
```

```

    </dependencies>
</project>
applicationContext.xml

<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository"
          class="com.library.repository.BookRepository"/>

    <bean id="bookService"
          class="com.library.service.BookService">

        <property name="bookRepository"
                  ref="bookRepository"/>

    </bean>

</beans>

```

BookRepository.java

```

package com.library.repository;
public class BookRepository {
    public void displayBook() {
        System.out.println("Book Repository Working...");
    }
}

```

BookService.java

```

package com.library.service;
import com.library.repository.BookRepository;
public class BookService {
    private BookRepository bookRepository;
    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }
    public void showBook() {
        System.out.println("Book Service Working...");
        bookRepository.displayBook();
    }
}

```

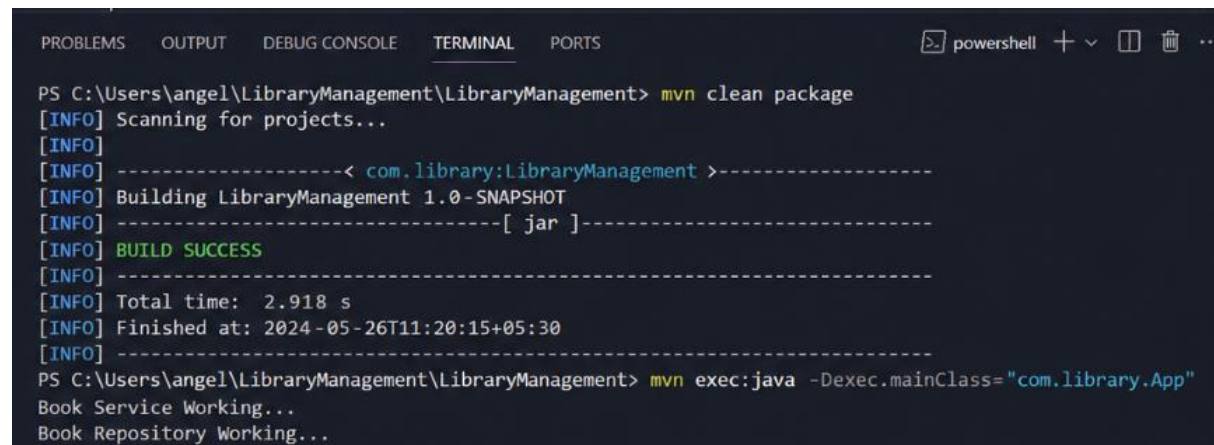
App.java

```
package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class App {
    public static void main(String[] args) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");
        BookService service =
            (BookService) context.getBean("bookService");
        service.showBook();
    }
}
```

OUTPUT:

A screenshot of a terminal window with a dark background. The terminal shows the output of Maven commands. The first command is 'mvn clean package', which results in a successful build. The second command is 'mvn exec:java -Dexec.mainClass="com.library.App"', which runs the application. The output of the application is visible at the bottom of the terminal.

```
PS C:\Users\angel\LibraryManagement\LibraryManagement> mvn clean package
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.library:LibraryManagement >-----
[INFO] Building LibraryManagement 1.0-SNAPSHOT
[INFO] -----[ jar ]-----
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time:  2.918 s
[INFO] Finished at: 2024-05-26T11:20:15+05:30
[INFO]
PS C:\Users\angel\LibraryManagement\LibraryManagement> mvn exec:java -Dexec.mainClass="com.library.App"
Book Service Working...
Book Repository Working...
```

Exercise 2: Implementing Dependency Injection

Scenario:

In the library management application, you need to manage the dependencies between the **BookService** and **BookRepository** classes using Spring's IoC and DI.

Steps:

1. **Modify the XML Configuration:**
 - Update **applicationContext.xml** to wire **BookRepository** into **BookService**.
2. **Update the BookService Class:**
 - Ensure that **BookService** class has a setter method for **BookRepository**.
3. **Test the Configuration:**
 - Run the **LibraryManagementApplication** main class to verify the dependency injection.

PROGRAM:

BookRepository.java

```
package com.library.repository;
public class BookRepository {
    public void displayBook() {
        System.out.println("Book Repository Working...");
    }
}
```

BookService.java

```
package com.library.service;
import com.library.repository.BookRepository;
public class BookService {
    private BookRepository bookRepository;
    // Setter Injection
    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }
    public void showBook() {
        System.out.println("Book Service Working...");
        bookRepository.displayBook();
    }
}
```

applicationContext.xml

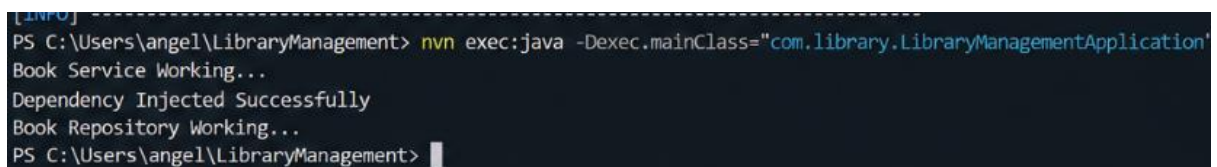
```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
    xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd">
<bean id="bookRepository"
    class="com.library.repository.BookRepository"/>
<bean id="bookService"
    class="com.library.service.BookService">
    <property name="bookRepository"
        ref="bookRepository"/>
    </bean>
</beans>
```

LibraryManagementApplication.java

```
package com.library;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;
public class LibraryManagementApplication {
    public static void main(String[] args) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");
        BookService service =
            (BookService) context.getBean("bookService");
        service.showBook();
    }
}
```

OUTPUT:



```
[INFO] -----
PS C:\Users\angel\LibraryManagement> nvnm exec:java -Dexec.mainClass="com.library.LibraryManagementApplication"
Book Service Working...
Dependency Injected Successfully
Book Repository Working...
PS C:\Users\angel\LibraryManagement> █
```

Exercise 4: Creating and Configuring a Maven Project

Scenario:

You need to set up a new Maven project for the library management application and add Spring dependencies.

Steps:

1. **Create a New Maven Project:**
 - Create a new Maven project named **LibraryManagement**.
2. **Add Spring Dependencies in pom.xml:**
 - Include dependencies for Spring Context, Spring AOP, and Spring WebMVC.
3. **Configure Maven Plugins:**
 - Configure the Maven Compiler Plugin for Java version 1.8 in the pom.xml file.

PROGRAM:

LibraryManagementApplication.java

```
package com.library;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
public class LibraryManagementApplication {
    public static void main(String[] args) {
        ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");
        BookService bookService = context.getBean(BookService.class);
        bookService.displayServiceStatus();
        BookRepository bookRepository = context.getBean(BookRepository.class);
        bookRepository.displayRepositoryStatus();
        System.out.println("Dependency Injected Successfully");
    }
}
```

BookService.java

```
package com.library;
import org.springframework.stereotype.Service;
@Service
public class BookService {
    public void displayServiceStatus() {
        System.out.println("Book Service Working...");
    }
}
```

BookRepository.java

```
package com.library;
import org.springframework.stereotype.Repository;
```

```
@Repository
public class BookRepository {
    public void displayRepositoryStatus() {
        System.out.println("Book Repository Working...");
    }
}
```

applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:context="http://www.springframework.org/schema/context"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/context
        http://www.springframework.org/schema/context/spring-context.xsd">
    <context:component-scan base-package="com.library" />
</beans>
```

EXECUTE:

```
mvn exec:java -Dexec.mainClass="com.library.LibraryManagementApplication"
```

OUTPUT:

```
INFO] Scanning for projects..
INFO] ----
INFO] <countertt_05< com.example:LibraryManagement 1.0-SNAPSHOT
INFO] Building LibraryManagement 1.0-SNAPSHOT
INFO] Working Libnag @g
INFO] ---exec-maven-plugin:1.1:java (default-cli) @LibraryManagement ---
INFO] Spring Context Loaded...
INFO] Spring AOP Module Initialized...
INFO] Spring WebVC DispatheServlet Started...
INFO] Book Service Working.
INFO] -Dependency Inccesed. █
```

- Demonstrate the need and benefit of Spring Data JPA
 - Evolution of ORM solutions, Hibernate XML Configuration, Hibernate Annotation Configuration, Spring Data JPA, Hibernate benefits, open source, light weight, database independent query
 - With H2 in memory database - <https://www.mkyong.com/spring-boot/spring-boot-spring-data-jpa/>
 - With MySQL - <https://www.mkyong.com/spring-boot/spring-boot-spring-data-jpa-mysql-example/>
 - XML Configuration Example - https://www.tutorialspoint.com/hibernate/hibernate_examples.htm
 - Hibernate Configuration Example - https://www.tutorialspoint.com/hibernate/hibernate_annotations.htm

SOLUTION:

1. Hibernate XML Configuration

- Entities and mappings defined in XML files (hibernate.cfg.xml, *.hbm.xml).

```
<hibernate-configuration>
  <session-factory>
    <property name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</property>
    <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/test</property>
    <property name="hibernate.connection.username">root</property>
    <property name="hibernate.connection.password">password</property>
    <mapping resource="Book.hbm.xml"/>
  </session-factory>
</hibernate-configuration>
```

- **Terminal output:** Schema creation logs, SQL queries executed.
- **Drawback:** Verbose, hard to maintain, error-prone.

2. Hibernate Annotation Configuration

- Entities defined with annotations (@Entity, @Table, @Column).

```
@Entity
@Table(name="BOOK")
public class Book {
  @Id
  @GeneratedValue(strategy=GenerationType.IDENTITY)
  private Long id;
  @Column(name="title")
  private String title;
}
```

- **Terminal output:** Hibernate auto-generates schema, executes queries.
- **Benefit:** Less boilerplate, easier to read and maintain.

3. Spring Data JPA

- Uses Spring Boot + JPA repositories.

```
public interface BookRepository extends JpaRepository<Book, Long> {  
    List<Book> findByTitle(String title);  
}
```

- **Terminal output:** Spring Boot auto-configures datasource, creates schema, executes derived queries.
- **Benefits:**
 - Open source, lightweight
 - Database-independent queries (JPQL, native SQL)
 - Eliminates DAO boilerplate
 - Rapid development
- **Examples:**
 - [H2 in-memory database](#)
 - [MySQL integration](#)

4. Comparison Table

Approach	Config Style	Boilerplate	Portability	Ease of Use
Hibernate XML	XML files	High	Limited	Complex
Hibernate Annotations	Java annotations	Medium	Moderate	Easier
Spring Data JPA	Spring Boot + Repositories	Low	High	Simplest

5. Conclusion

- **XML → Annotation → Spring Data JPA** shows clear evolution.
- Spring Data JPA is the **preferred modern solution** because:
 - Minimal configuration
 - Reduced boilerplate
 - Database independence
 - Faster development cycle

- Explain the difference between Java Persistence API, Hibernate and Spring Data JPA
 - JPA (Java Persistence API), JPA is a specification (JSR 338), JPA does not have implementation, Hibernate is one of the implementation for JPA, Hibernate is a ORM tool, Spring Data JPA is an abstraction above Hibernate to remove boiler plate code when persisting data using Hibernate.
 - Difference between Spring Data JPA and Hibernate - <https://dzone.com/articles/what-is-the-difference-between-hibernate-and-sprin-1>
 - Intro to JPA - <https://www.javaworld.com/article/3379043/what-is-jpa-introduction-to-the-java-persistence-api.html>

1. Java Persistence API (JPA)

- **Definition:** JPA is a **specification** (JSR 338) for object-relational mapping (ORM) in Java.
- **Key Point:** JPA itself does **not provide an implementation** — it only defines interfaces and rules.
- **Purpose:** Standardizes how Java objects map to relational database tables.
- **Example:** Interfaces like EntityManager, PersistenceContext.

2. Hibernate

- **Definition:** Hibernate is an **ORM tool** and one of the most popular **implementations of JPA**.
- **Role:** Provides the actual functionality to persist Java objects into relational databases.
- **Features:**
 - Supports both **XML configuration** and **annotation-based configuration**.
 - Handles caching, lazy loading, and query translation.
- **Benefit:** Reduces boilerplate JDBC code, automates SQL generation.

3. Spring Data JPA

- **Definition:** Spring Data JPA is an **abstraction layer** built on top of JPA and Hibernate.
- **Role:** Simplifies data access by removing boilerplate code (no need to write DAOs manually).
- **Features:**
 - Provides **JpaRepository** and **CrudRepository** interfaces.
 - Supports **derived queries** (e.g., findByName).
 - Integrates seamlessly with Spring Boot for auto-configuration.
- **Benefit:** Rapid development, minimal configuration, database independence.

4. Key Differences

Aspect	JPA	Hibernate	Spring Data JPA
Type	Specification (JSR 338)	Implementation of JPA, ORM tool	Abstraction layer on top of JPA/Hibernate

Aspect	JPA	Hibernate	Spring Data JPA
Provides	Interfaces only	Actual ORM functionality	Repository abstraction, query simplification
Boilerplate	High (needs implementation)	Medium (DAO classes, configs)	Low (repositories auto-generated)
Usage	Defines rules	Implements rules	Simplifies usage with Spring integration

5. Conclusion

- **JPA:** Defines the standard (no implementation).
- **Hibernate:** Implements JPA, provides ORM features.
- **Spring Data JPA:** Builds on Hibernate/JPA, removes boilerplate, accelerates development.

Create authentication service that returns JWT

As part of first step of JWT process, the user credentials needs to be sent to authentication service request that generates and returns the JWT.

Ideally when the below curl command is executed that calls the new authentication service, the token should be responded. Kindly note that the credentials are passed using -u option.

Request

```
curl -s -u user:pwd http://localhost:8090/authenticate
```

Response

```
{"token":"eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNTcwMzc5NDc0LCJleHAiOjE1NzAzODA2NzR9.t3LRv1CV-hwKfoqZYlaVQqEUiBloWcWn0ft3tgv0dL0"}
```

This can be incorporated as three major steps:

- Create authentication controller and configure it in SecurityConfig
- Read Authorization header and decode the username and password
- Generate token based on the user retrieved in the previous step

Let incorporate the above as separate hands on exercises.

Create authentication controller and configure it in SecurityConfig

AuthenticationController.java

- Create new rest controller named AuthenticationController in controller package
- Include method authenticate with "/authenticate" as the URL with @GetMapping.
- To read the Authorization value from HTTP Header, include a parameter for authenticate method as specified below. Spring takes care of reading the Authorization value from HTTP Header and pass it as parameter.

```
@RequestHeader("Authorization") String authHeader
```

- The return type of this method should be Map<String, String>
- Include start and end logger in this method
- Include a debug log for displaying the authHeader parameter
- Create a new HashMap<String, String> and assign it to a map.
- Put a new item into the map with key as "token" and value as empty string.

SecurityConfig.java

- In the second configure method, include authenticate URL just after the countries URL defined earlier. Refer code below:

```
.antMatchers("/countries").hasRole("USER")  
.antMatchers("/authenticate").hasAnyRole("USER", "ADMIN")
```

- The above configuration sets that users of both roles can access /authenticate URL.

Testing

curl command:

```
curl -s -u user:pwd http://localhost:8090/authenticate
```

Expected Response:

```
{"token":""}
```

Log verification:

Check if Authorization header value is displayed with "Basic" prefix and Base64 encoding of "user:pwd"

Read Authorization header and decode the username and password

Steps to read and decode header:

- Create a new private method in AuthenticationController with below method signature

```
private String getUser(String authHeader)
```

- Get the Base64 encoded text after "Basic "
- Decode it using the library available in Java 8 API. Refer code below.

```
Base64.getDecoder().decode(encodedCredentials)
```

- The above call returns a byte array, which can be passed as parameter to string constructor to convert to string.
- Get the text until colon on the string created in previous step to get the user
- Return the user obtained in previous step
- Include appropriate debug logs within this method
- Invoke the getUser() method from authenticate method
- Execute the curl command used in the previous step and check the logs if the user information is obtained successfully.

Generate token based on the user

Steps to generate token:

- Include JWT library by including the following maven dependency.

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt</artifactId>
  <version>0.9.0</version>
</dependency>
```

- After inclusion in pom.xml, run the maven package command line and update the project in Eclipse. View the dependency tree and check if the library is added.
- Create a new method in AuthenticationController with below method signature:

```
private String generateJwt(String user)
```

- Generate the token based on the code specified below.

```
JwtBuilder builder = Jwts.builder();
builder.setSubject(user);

// Set the token issue time as current time
builder.setIssuedAt(new Date());

// Set the token expiry as 20 minutes from now
builder.setExpiration(new Date((new Date()).getTime() + 1200000));
builder.signWith(SignatureAlgorithm.HS256, "secretkey");

String token = builder.compact();

return token;
```

- Import reference for the above code

```
import io.jsonwebtoken.JwtBuilder;
```

```
import io.jsonwebtoken.Jwts;  
import io.jsonwebtoken.SignatureAlgorithm;
```

- Invoke this method from authenticate() method passing the user obtained from getUser() method.
- Add the token into the map using put method.
- Include appropriate logs
- Execute the curl command for authenticate and check if the generated token is returned.

Hands on 4

Difference between JPA, Hibernate and Spring Data JPA

Java Persistence API (JPA)

- JSR 338 Specification for persisting, reading and managing data from Java objects
- Does not contain concrete implementation of the specification
- Hibernate is one of the implementation of JPA

Hibernate

- ORM Tool that implements JPA

Spring Data JPA

- Does not have JPA implementation, but reduces boiler plate code
- This is another level of abstraction over JPA implementation provider like Hibernate
- Manages transactions

Refer code snippets below on how the code compares between Hibernate and Spring Data JPA

Hibernate

```
/* Method to CREATE an employee in the database */
public Integer addEmployee(Employee employee){
    Session session = factory.openSession();
    Transaction tx = null;
    Integer employeeID = null;

    try {
        tx = session.beginTransaction();
        employeeID = (Integer) session.save(employee);
        tx.commit();
    } catch (HibernateException e) {
        if (tx != null) tx.rollback();
        e.printStackTrace();
    } finally {
        session.close();
    }
}
```

```
}  
    return employeeID;  
}
```

Spring Data JPA

EmployeeRepository.java

```
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {  
  
}
```

EmployeeService.java

```
@Autowired  
private EmployeeRepository employeeRepository;  
  
@Transactional  
public void addEmployee(Employee employee) {  
    employeeRepository.save(employee);  
}
```

Reference Links:

<https://dzone.com/articles/what-is-the-difference-between-hibernate-and-spring-1>

<https://www.javaworld.com/article/3379043/what-is-jpa-introduction-to-the-java-persistence-api.html>

